

DOCUMENTO DE REQUERIMIENTOS DE SOFTWARE

Aplicación: Cancionero Litúrgico

Versión: 1.0

Fecha: 14 de Octubre de 2025

Equipo: Ingeniería de Software

Estado: Aprobado para desarrollo

1. RESUMEN EJECUTIVO

1.1 Descripción del Proyecto

Desarrollo de una aplicación móvil Android para la gestión y visualización de canciones litúrgicas con acordes de guitarra. La aplicación está orientada a músicos de iglesia que necesitan organizar repertorios, transponer acordes y visualizar canciones durante las celebraciones litúrgicas.

1.2 Objetivos

- Centralizar el repertorio de canciones litúrgicas del grupo musical
- Facilitar la visualización de letras y acordes durante las celebraciones
- Permitir transposición de acordes y gestión de capotraste
- Organizar canciones en setlists por evento
- Funcionar completamente offline con sincronización inicial desde BD central

1.3 Alcance

Incluido en esta versión:

- Gestión completa de canciones (CRUD)
- Sistema de categorización múltiple
- Creación y gestión de setlists
- Transposición de acordes y gestión de capotraste
- Modo presentación optimizado para ejecución
- Búsqueda y filtrado avanzado
- Sincronización unidireccional con BD central
- Importación/exportación de canciones
- Scroll automático inteligente basado en tempo

Excluido de esta versión:

- Detección de acordes por audio (función futura)
- Sincronización en tiempo real entre dispositivos
- Edición colaborativa simultánea
- Generación de partituras formales

2. REQUERIMIENTOS FUNCIONALES

2.1 GESTIÓN DE CANCIONES

RF-001: Crear Canción

Prioridad: Alta

Descripción: El sistema debe permitir la creación de nuevas canciones con todos sus metadatos.

Criterios de aceptación:

- El usuario puede crear una canción con los siguientes campos:
 - Título (obligatorio, máx. 200 caracteres)
 - Autor/Compositor (opcional, máx. 100 caracteres)
 - Letra con acordes integrados (obligatorio, sin límite)
 - Tonalidad original (obligatorio, formato: C, D, Em, etc.)
 - Tempo en BPM (opcional, rango: 40-240)
 - Posición de capo (opcional, rango: 0-12)
 - Enlaces a videos (opcional, múltiples URLs)
 - Notas/observaciones (opcional, máx. 500 caracteres)
- El sistema genera automáticamente fecha de creación
- El sistema valida que el título no esté vacío
- El sistema guarda la canción en la base de datos local
- Se muestra mensaje de confirmación al guardar exitosamente

Notas técnicas:

- Los acordes deben poder insertarse directamente sobre la letra
 - Soportar saltos de línea y espaciado personalizado
-

RF-002: Editar Canción

Prioridad: Alta

Descripción: El sistema debe permitir la modificación de canciones existentes.

Criterios de aceptación:

- El usuario puede modificar cualquier campo de una canción existente
 - El sistema actualiza automáticamente la fecha de última modificación
 - Los cambios se reflejan inmediatamente en todas las vistas
 - Si la canción está en setlists, los cambios se propagan automáticamente
 - Se muestra mensaje de confirmación al guardar cambios
-

RF-003: Eliminar Canción

Prioridad: Alta

Descripción: El sistema debe permitir la eliminación de canciones.

Criterios de aceptación:

- El usuario puede eliminar una canción desde la vista de detalle
- El sistema muestra diálogo de confirmación antes de eliminar
- El diálogo indica si la canción está en setlists activos
- Al confirmar, la canción se elimina de:
 - Base de datos principal
 - Todas las categorías

- Todos los setlists
 - Se muestra mensaje de confirmación tras la eliminación
 - La acción no es reversible (sin undo)
-

RF-004: Insertar Acordes sobre Letra

Prioridad: Alta

Descripción: El sistema debe permitir insertar acordes en notación americana y latina sobre el texto de la letra.

Criterios de aceptación:

- El usuario puede insertar acordes en cualquier posición de la letra
- Soporta notación americana: C, D, E, F, G, A, B
- Soporta notación latina: Do, Re, Mi, Fa, Sol, La, Si
- Soporta modificadores: # (sostenido), b (bemol), m (menor)
- Soporta extensiones: 7, 9, 11, 13, maj7, m7, sus2, sus4, dim, aug, add9
- Los acordes se posicionan exactamente sobre la sílaba correspondiente
- Los acordes se distinguen visualmente del texto (color diferente)
- El editor muestra preview en tiempo real

Formato de entrada sugerido:



C Am F G

Ejemplo de letra con acordes posicionados

RF-005: Transponer Acordes

Prioridad: Alta

Descripción: El sistema debe permitir transponer todos los acordes de una canción en semitonos.

Criterios de aceptación:

- El usuario puede transponer en rango de -11 a +11 semitonos
- Se muestra control de transposición en la vista de detalle y modo presentación
- Al transponer:
 - Todos los acordes de la canción se actualizan automáticamente
 - Se mantiene el tipo de acorde (mayores, menores, extensiones)
 - Se muestra la diferencia en semitonos respecto al tono original
 - Se muestra tanto el tono original como el tono actual
- La transposición no modifica la canción original (es temporal)
- Se puede resetear a tono original con un botón
- Funciona con ambas notaciones (americana y latina)

Ejemplo:

- Original: C - Am - F - G
 - Transponer +2: D - Bm - G - A
-

RF-006: Gestión de Capotraste

Prioridad: Alta

Descripción: El sistema debe permitir configurar la posición del capo y recalcular acordes en consecuencia.

Criterios de aceptación:

- El usuario puede configurar posición de capo (0-12 trastes)
- Al cambiar la posición del capo:
 - Los acordes mostrados se ajustan automáticamente
 - Se mantiene la tonalidad sonora real
 - Se muestra claramente la posición del capo activa
- Se puede guardar una posición de capo predeterminada por canción
- Se muestra indicador visual: "Capo: X" o "Sin capo"
- En modo presentación, el indicador de capo es prominente

Ejemplo:

- Canción en G con Capo 2: se muestran acordes en F
- Sonido real: G, acordes a tocar: F

2.2 ORGANIZACIÓN Y BÚSQUEDA

RF-007: Sistema de Categorización Múltiple

Prioridad: Alta

Descripción: El sistema debe permitir asignar múltiples categorías a cada canción y gestionar categorías personalizadas.

Criterios de aceptación:

- El sistema incluye categorías predefinidas:
 - Entrada
 - Meditación
 - Virgen María
 - Comunión
 - Ofertorio
 - Salida
 - Adoración
 - Penitencial
 - Aleluya
- El usuario puede crear categorías personalizadas
- El usuario puede asignar múltiples categorías a una canción
- El usuario puede editar nombre de categorías personalizadas
- El usuario puede eliminar categorías (con confirmación)
- Al eliminar una categoría, las canciones no se eliminan
- Se muestra contador de canciones por categoría

RF-008: Marcar Favoritos

Prioridad: Media

Descripción: El sistema debe permitir marcar canciones como favoritas para acceso rápido.

Criterios de aceptación:

- El usuario puede marcar/desmarcar canciones como favoritas con un ícono
- Existe una vista "Favoritos" que muestra solo canciones marcadas
- El estado de favorito se persiste en la base de datos
- Se muestra indicador visual en listados (ícono de estrella)
- El cambio de estado es inmediato (sin confirmación)

RF-009: Listado Alfabético

Prioridad: Alta

Descripción: El sistema debe mostrar canciones ordenadas alfabéticamente con navegación rápida.

Criterios de aceptación:

- Vista principal muestra canciones ordenadas alfabéticamente por título
 - Se incluye índice alfabético lateral para navegación rápida (A-Z)
 - Al tocar una letra del índice, salta a la primera canción con esa inicial
 - Se ignoran artículos al ordenar (ej: "El Señor" se ordena como "Señor")
 - Se muestran mínimo: título, primera línea de letra, categorías
-

RF-010: Búsqueda por Texto

Prioridad: Alta

Descripción: El sistema debe permitir búsqueda en tiempo real por múltiples campos.

Criterios de aceptación:

- Campo de búsqueda siempre visible en la parte superior
 - La búsqueda es en tiempo real (mientras se escribe)
 - Busca en los siguientes campos:
 - Título de la canción
 - Autor/Compositor
 - Contenido completo de la letra
 - Notas/observaciones
 - Se muestran resultados filtrados instantáneamente
 - Se resalta el texto coincidente en los resultados
 - Búsqueda insensible a mayúsculas y acentos
 - Se muestra mensaje "No se encontraron resultados" si corresponde
 - Botón para limpiar búsqueda rápidamente
-

RF-011: Filtrar por Categoría

Prioridad: Media

Descripción: El sistema debe permitir filtrar canciones por una o múltiples categorías.

Criterios de aceptación:

- Existe una vista de filtros donde se muestran todas las categorías
 - El usuario puede seleccionar múltiples categorías simultáneamente
 - Se muestra el número de canciones por categoría
 - Los filtros se aplican con operador OR (muestra canciones que tengan AL MENOS una categoría seleccionada)
 - Se puede combinar filtro de categoría con búsqueda de texto
 - Se muestra indicador visual de filtros activos
 - Botón para limpiar todos los filtros
-

2.3 LISTAS DE REPRODUCCIÓN (SETLISTS)

RF-012: Crear Setlist

Prioridad: Alta

Descripción: El sistema debe permitir crear listas de reproducción para eventos específicos.

Criterios de aceptación:

- El usuario puede crear un nuevo setlist con:
 - Nombre (obligatorio, máx. 100 caracteres)
 - Fecha del evento (opcional)
 - Notas generales (opcional, máx. 300 caracteres)
 - El sistema genera fecha de creación automáticamente
 - Se muestra el setlist vacío listo para agregar canciones
 - Se valida que el nombre no esté vacío
 - Se muestra mensaje de confirmación al crear
-

RF-013: Agregar Canciones a Setlist

Prioridad: Alta

Descripción: El sistema debe permitir agregar canciones del repertorio a un setlist.

Criterios de aceptación:

- El usuario puede agregar canciones mediante:
 - Búsqueda dentro del selector de canciones
 - Lista completa del repertorio
 - Se puede agregar la misma canción múltiples veces al mismo setlist
 - Cada instancia puede tener:
 - Transposición personalizada diferente
 - Posición de capo personalizada diferente
 - Las canciones se agregan al final de la lista por defecto
 - Se muestra confirmación visual al agregar
 - El selector de canciones se cierra automáticamente tras agregar
-

RF-014: Reordenar Canciones en Setlist

Prioridad: Alta

Descripción: El sistema debe permitir cambiar el orden de las canciones dentro de un setlist.

Criterios de aceptación:

- El usuario puede reordenar canciones mediante drag & drop
 - Opcionalmente, botones de "mover arriba" / "mover abajo"
 - El nuevo orden se guarda automáticamente
 - Se muestra numeración/orden actual (1, 2, 3...)
 - La reordenación es fluida y sin lag
-

RF-015: Editar Setlist

Prioridad: Media

Descripción: El sistema debe permitir modificar las propiedades de un setlist.

Criterios de aceptación:

- El usuario puede editar:
 - Nombre del setlist
 - Fecha del evento
 - Notas generales
- El usuario puede eliminar canciones individuales del setlist sin eliminarlas del repertorio
- Los cambios se guardan automáticamente o con botón explícito
- Se muestra fecha de última modificación

RF-016: Eliminar Setlist

Prioridad: Media

Descripción: El sistema debe permitir eliminar setlists completos.

Criterios de aceptación:

- El usuario puede eliminar un setlist completo
 - Se muestra diálogo de confirmación con el nombre del setlist
 - Al confirmar, se elimina el setlist y todas sus asociaciones
 - Las canciones del repertorio NO se eliminan
 - Se muestra mensaje de confirmación tras eliminar
 - La acción no es reversible
-

RF-017: Reproducir Setlist

Prioridad: Alta

Descripción: El sistema debe permitir reproducir un setlist completo con navegación entre canciones.

Criterios de aceptación:

- Desde un setlist, el usuario puede iniciar el modo presentación
 - Se muestra la primera canción del setlist automáticamente
 - Controles de navegación permiten:
 - Avanzar a la siguiente canción
 - Retroceder a la canción anterior
 - Ver indicador de posición (ej: "3 de 8")
 - Al llegar a la última canción, se muestra indicador
 - Se puede salir del modo reproducción en cualquier momento
 - Se respetan transposiciones y capos personalizados de cada canción en el setlist
-

2.4 VISUALIZACIÓN Y MODO PRESENTACIÓN

RF-018: Modo Presentación

Prioridad: Alta

Descripción: El sistema debe ofrecer un modo de visualización optimizado para ejecución en vivo.

Criterios de aceptación:

- El modo presentación muestra:
 - Letra de la canción en fuente grande
 - Acordes destacados visualmente (color distintivo, negritas)
 - Título de la canción en la parte superior
 - Tonalidad actual y capo (si aplica)
- Elementos ocultos en modo presentación:
 - Barra de navegación
 - Menús
 - Botones de edición
- Controles mínimos visibles:
 - Botón de salida (X o back)
 - Controles de navegación (si es parte de un setlist)
 - Control de tamaño de fuente
 - Control de transposición
 - Botón de scroll automático

- La pantalla utiliza todo el espacio disponible
 - Entrada y salida del modo son fluidas
-

RF-019: Tamaño de Fuente Ajustable

Prioridad: Alta

Descripción: El sistema debe permitir ajustar el tamaño de fuente en tiempo real.

Criterios de aceptación:

- Control de tamaño de fuente visible en modo presentación
 - Mínimo 5 niveles de tamaño (Muy pequeño, Pequeño, Mediano, Grande, Muy grande)
 - El ajuste se aplica inmediatamente sin recargar
 - El tamaño seleccionado se guarda como preferencia global del usuario
 - Los acordes mantienen proporcionalidad con el texto
 - El cambio de tamaño no afecta el scroll automático en curso
-

RF-020: Modo Nocturno/Claro

Prioridad: Alta

Descripción: El sistema debe ofrecer temas visuales para diferentes condiciones de iluminación.

Criterios de aceptación:

- El sistema ofrece dos temas:
 - **Modo Claro:** Fondo blanco/claro, texto negro/oscura
 - **Modo Nocturno:** Fondo negro/oscura, texto blanco/claro
 - El cambio de tema se puede hacer desde:
 - Configuración general
 - Modo presentación (botón de acceso rápido)
 - El cambio se aplica inmediatamente en toda la aplicación
 - La preferencia se guarda y persiste entre sesiones
 - Los acordes mantienen buen contraste en ambos modos
 - Los colores de categorías se ajustan según el tema
-

RF-021: Scroll Automático Inteligente

Prioridad: Alta

Descripción: El sistema debe ofrecer scroll automático basado en el tempo de la canción.

Criterios de aceptación:

- Botón de scroll automático visible en modo presentación
- Si la canción tiene tempo definido:
 - El scroll se calcula automáticamente según BPM
 - La velocidad es proporcional al tempo musical
 - Se ajusta a la longitud total de la canción
- Si la canción NO tiene tempo:
 - Se usa velocidad predeterminada (configurable)
- Controles de scroll automático:
 - Play/Pause con un toque en la pantalla
 - Ajuste manual de velocidad +/- (independiente del tempo)
 - Botón de stop para resetear al inicio
- El scroll es suave y constante (sin saltos)
- Al pausar, mantiene la posición actual
- Al llegar al final, se detiene automáticamente

- Indicador visual del estado (reproduciendo/pausado)

Cálculo sugerido:

- Velocidad base = $(\text{BPM} / 60) * \text{factor_ajuste}$
 - El factor_ajuste es configurable por el usuario
-

RF-022: Mantener Pantalla Activa

Prioridad: Alta

Descripción: El sistema debe prevenir que la pantalla se apague durante el modo presentación.

Criterios de aceptación:

- Al entrar en modo presentación, se activa el wake lock
 - La pantalla permanece encendida incluso sin interacción
 - Al salir del modo presentación, se desactiva el wake lock automáticamente
 - No consume batería innecesaria fuera del modo presentación
 - Funciona en cualquier configuración de ahorro de energía del dispositivo
-

2.5 SINCRONIZACIÓN Y BASE DE DATOS CENTRAL

RF-023: Sincronización Inicial con BD Central

Prioridad: Alta

Descripción: El sistema debe sincronizar el contenido con una base de datos central al abrir la aplicación.

Criterios de aceptación:

- Al abrir la aplicación, se verifica conexión a BD central
- Si hay conexión:
 - Se descargan canciones nuevas o modificadas desde la última sincronización
 - Se descargan categorías nuevas
 - Se descargan setlists compartidos (si existen)
 - Se muestra progreso de la sincronización
 - Se notifica al usuario el resultado (X canciones nuevas, Y actualizadas)
- Si NO hay conexión:
 - La app funciona con datos locales
 - Se muestra indicador de "Sin sincronización"
 - No se bloquea ninguna funcionalidad
- La sincronización es unidireccional: Central → Local
- Las canciones locales NO se suben a la BD central (sin write-back)
- Se registra timestamp de última sincronización
- Opción de sincronizar manualmente desde configuración

Notas técnicas:

- Implementar sistema de versionado o timestamps para detectar cambios
 - Manejo de conflictos: los datos de la BD central sobrescriben los locales
 - Considerar sincronización incremental (solo cambios, no todo)
-

RF-024: Indicador de Estado de Sincronización

Prioridad: Media

Descripción: El sistema debe informar al usuario sobre el estado de sincronización.

Criterios de aceptación:

- Se muestra indicador en la pantalla principal:
 - Ícono de sincronización exitosa con timestamp
 - Ícono de error si falló la última sincronización
 - Ícono de sin conexión si está offline
 - Al tocar el indicador, se muestra detalle:
 - Fecha y hora de última sincronización
 - Cantidad de elementos sincronizados
 - Errores si los hubo
 - El indicador se actualiza automáticamente tras cada sincronización
-

2.6 IMPORTACIÓN Y EXPORTACIÓN

RF-025: Exportar Canción Individual

Prioridad: Media

Descripción: El sistema debe permitir exportar canciones individuales para compartir.

Criterios de aceptación:

- Desde la vista de detalle de canción, opción "Exportar"
 - Formato de exportación: JSON estructurado
 - El archivo incluye:
 - Todos los campos de la canción
 - Categorías asociadas
 - Metadatos de exportación (fecha, versión)
 - Se puede compartir mediante:
 - WhatsApp, Email, Telegram, etc.
 - Guardado en almacenamiento local
 - Nombre de archivo sugerido: "NombreCancion_YYYYMMDD.json"
-

RF-026: Importar Canción Individual

Prioridad: Media

Descripción: El sistema debe permitir importar canciones desde archivos.

Criterios de aceptación:

- Opción "Importar canción" en menú principal
 - Soporta importación desde:
 - Selector de archivos del sistema
 - Intención de compartir (share intent) desde otras apps
 - Valida formato del archivo antes de importar
 - Si el formato es inválido, muestra error descriptivo
 - Detecta duplicados por título:
 - Si existe, pregunta: "¿Sobrescribir canción existente?"
 - Opciones: Sobrescribir / Importar como nueva / Cancelar
 - Si incluye categorías no existentes, las crea automáticamente
 - Muestra mensaje de confirmación tras importar exitosamente
-

RF-027: Exportar Setlist Completo

Prioridad: Media

Descripción: El sistema debe permitir exportar setlists con todas sus canciones.

Criterios de aceptación:

- Desde la vista de detalle de setlist, opción "Exportar"
 - Formato de exportación: JSON estructurado
 - El archivo incluye:
 - Metadatos del setlist (nombre, fecha, notas)
 - Array completo de canciones con todos sus datos
 - Orden de las canciones
 - Transposiciones y capos personalizados por canción
 - Se puede compartir mediante las apps estándar del sistema
 - Nombre de archivo sugerido: "Setlist_NombreEvento_YYYYMMDD.json"
-

RF-028: Importar Setlist Completo

Prioridad: Media

Descripción: El sistema debe permitir importar setlists con todas sus canciones.

Criterios de aceptación:

- Opción "Importar setlist" en menú de setlists
 - Valida formato del archivo antes de importar
 - Proceso de importación:
 1. Lee el archivo y valida estructura
 2. Identifica canciones faltantes en el repertorio local
 3. Muestra resumen: "X canciones ya existen, Y se importarán"
 4. Usuario confirma importación
 5. Se importan las canciones faltantes primero
 6. Se crea el setlist con todas las canciones y configuraciones
 - Si el nombre del setlist existe, sugiere nombre alternativo
 - Muestra mensaje de confirmación con resumen
-

RF-029: Backup Completo de la Aplicación

Prioridad: Media

Descripción: El sistema debe permitir crear un backup completo de todos los datos.

Criterios de aceptación:

- Opción "Crear backup" en configuración
 - El backup incluye:
 - Todas las canciones
 - Todas las categorías
 - Todos los setlists
 - Configuraciones de usuario (preferencias de visualización, etc.)
 - Formato: Archivo ZIP con estructura interna en JSON
 - Nombre de archivo: "CancioneroBackup_YYYYMMDD_HHMMSS.zip"
 - Se puede guardar en almacenamiento local o compartir
 - Muestra progreso durante la creación del backup
 - Mensaje de confirmación al completar
-

RF-030: Restaurar desde Backup

Prioridad: Media

Descripción: El sistema debe permitir restaurar todos los datos desde un backup.

Criterios de aceptación:

- Opción "Restaurar backup" en configuración
 - Permite seleccionar archivo de backup desde el sistema de archivos
 - Valida integridad del archivo de backup
 - Muestra advertencia clara:
 - "Esta acción sobrescribirá todos los datos actuales"
 - "No se puede deshacer"
 - Requiere confirmación explícita del usuario
 - Proceso de restauración:
 1. Valida archivo
 2. Crea backup automático de datos actuales (por seguridad)
 3. Limpia la base de datos actual
 4. Importa todos los datos del backup
 5. Reinicia la aplicación o recarga vistas
 - Muestra progreso durante la restauración
 - En caso de error, intenta revertir usando el backup automático
 - Mensaje de confirmación al completar exitosamente
-

RF-031: Importar Canciones desde Websites (Plus)

Prioridad: Baja (Plus)

Descripción: El sistema debe permitir importar canciones desde sitios web especializados en acordes.

Criterios de aceptación:

- Opción "Importar desde web" en menú de importación
- El usuario ingresa URL de la canción
- Sitios soportados (ejemplos):
 - Cifra Club
 - Ultimate Guitar
 - Acordes de Música Cristiano
 - Otros sitios con formato similar
- El sistema:
 1. Descarga el contenido HTML
 2. Parsea y extrae: título, autor, letra, acordes
 3. Muestra preview de la canción importada
 4. Usuario puede editar antes de confirmar
 5. Al confirmar, se guarda en el repertorio
- Manejo de errores:
 - URL inválida
 - Sitio no soportado
 - Error de conexión
 - Formato no reconocido
- Se muestra mensaje descriptivo en cada caso

Notas técnicas:

- Implementar web scraping con HTML parsing
 - Considerar cambios en estructura HTML de sitios externos
 - Implementar adaptadores por sitio web específico
 - Respetar términos de uso de sitios externos
-

2.7 CONFIGURACIÓN Y PREFERENCIAS

RF-032: Configuración de Usuario

Prioridad: Media

Descripción: El sistema debe permitir personalizar preferencias de la aplicación.

Criterios de aceptación:

- Vista de configuración accesible desde menú principal
 - Configuraciones disponibles:
 - **Visualización:**
 - Tema (Claro/Nocturno)
 - Tamaño de fuente predeterminado
 - Notación de acordes preferida (Americana/Latina/Ambas)
 - **Scroll Automático:**
 - Velocidad predeterminada (Lento/Medio/Rápido)
 - Factor de ajuste del tempo (slider 0.5x - 2.0x)
 - **Sincronización:**
 - URL de BD central
 - Sincronizar automáticamente al abrir (On/Off)
 - **Sobre la App:**
 - Versión
 - Información de desarrollo
 - Todas las preferencias se guardan localmente
 - Cambios se aplican inmediatamente
-

2.8 INTERFAZ Y NAVEGACIÓN

RF-033: Navegación Principal

Prioridad: Alta

Descripción: El sistema debe ofrecer navegación intuitiva entre las secciones principales.

Criterios de aceptación:

- Bottom navigation bar o drawer navigation con secciones:
 1. **Canciones** (vista principal)
 2. **Setlists**
 3. **Categorías**
 4. **Favoritos**
 5. **Configuración**
 - La sección activa se indica visualmente
 - Transiciones suaves entre secciones
 - Botón FAB para agregar canción/setlist según contexto
-

RF-034: Vista de Detalle de Canción

Prioridad: Alta

Descripción: El sistema debe mostrar todos los detalles de una canción de forma organizada.

Criterios de aceptación:

- Al seleccionar una canción, se abre vista de detalle que muestra:
 - Título (destacado)
 - Autor
 - Categorías (chips)
 - Tonalidad y capo
 - Tempo
 - Letra completa con acordes
 - Notas
 - Enlaces a videos (como botones)
- Acciones disponibles:
 - Editar

- Eliminar
 - Exportar
 - Agregar a setlist
 - Marcar/desmarcar favorito
 - Iniciar modo presentación
 - Controles de transposición y capo visibles
 - Vista optimizada para lectura
-

3. REQUERIMIENTOS NO FUNCIONALES

3.1 RENDIMIENTO

RNF-001: Tiempo de Inicio

Descripción: La aplicación debe iniciar rápidamente.
Métrica: Tiempo desde tap en ícono hasta vista principal completamente cargada ≤ 3 segundos.
Condiciones: En dispositivos Android 8.0+ con especificaciones medias, con repertorio de hasta 500 canciones.

RNF-002: Tiempo de Búsqueda

Descripción: La búsqueda debe mostrar resultados instantáneamente.
Métrica: Resultados de búsqueda visibles en ≤ 500 ms tras escribir cada carácter.
Condiciones: Con repertorio de hasta 500 canciones.

RNF-003: Navegación Fluida

Descripción: El cambio entre canciones y vistas debe ser inmediato.
Métrica: Transición entre canciones en modo presentación ≤ 200 ms. Navegación entre vistas principales ≤ 300 ms.
Condiciones: Sin lag perceptible, animaciones a 60 FPS.

RNF-004: Scroll Automático Suave

Descripción: El scroll automático debe ser fluido sin saltos.
Métrica: Animación a 60 FPS constante, sin frame drops.
Condiciones: En canciones de cualquier longitud.

RNF-005: Capacidad de Almacenamiento

Descripción: El sistema debe manejar grandes repertorios sin degradación.
Métrica: Soporte para mínimo 500 canciones sin impacto perceptible en rendimiento.
Objetivo extendido: 1000+ canciones.

3.2 USABILIDAD

RNF-006: Interfaz Intuitiva

Descripción: La aplicación debe ser fácil de usar sin capacitación previa.
Criterios:

- Usuarios músicos sin experiencia técnica pueden realizar tareas básicas sin ayuda

- Iconografía estándar de Material Design
 - Feedback visual inmediato para todas las acciones
 - Mensajes de error claros y accionables
 - Flujos de trabajo no requieren más de 3 toques para acciones comunes
-

RNF-007: Accesibilidad con Una Mano

Descripción: Las funciones críticas deben ser operables con una mano durante la ejecución.

Criterios:

- Botones principales en zona de alcance del pulgar
 - Botones de mínimo 48dp × 48dp (Material Design estándar)
 - Controles de modo presentación fácilmente alcanzables
 - Gestos simples: tap, doble tap, swipe (sin gestos complejos)
-

RNF-008: Legibilidad

Descripción: El contenido debe ser fácilmente legible en condiciones variables.

Criterios:

- Contraste mínimo WCAG AA (4.5:1 para texto normal)
 - Tamaño de fuente mínimo en modo presentación: 18sp
 - Los acordes deben distinguirse claramente de la letra
 - Funcionamiento en diferentes niveles de brillo ambiental
-

RNF-009: Retroalimentación de Acciones

Descripción: El usuario debe recibir confirmación de todas las acciones importantes.

Criterios:

- Snackbars o toasts para confirmaciones
 - Diálogos modales para acciones destructivas
 - Indicadores de progreso para operaciones largas (>1 segundo)
 - Estados de carga visibles durante sincronización
-

3.3 DISPONIBILIDAD Y CONFIABILIDAD

RNF-010: Funcionamiento Offline

Descripción: La aplicación debe funcionar completamente sin conexión a internet.

Criterios:

- 100% de funcionalidad disponible offline excepto:
 - Sincronización con BD central
 - Importación desde websites
 - Reproducción de videos externos (YouTube)
 - Todas las canciones, setlists y configuraciones accesibles sin internet
 - No mostrar errores relacionados con conectividad durante uso normal offline
-

RNF-011: Persistencia de Datos

Descripción: Los datos del usuario deben persistir de forma confiable.

Criterios:

- Todas las modificaciones se guardan en SQLite inmediatamente
 - Los datos persisten tras:
 - Cierre de la aplicación
 - Reinicio del dispositivo
 - Actualizaciones de la app
 - Transacciones atómicas en base de datos (no se corrompe por cierres inesperados)
 - Integridad referencial garantizada (foreign keys, constraints)
-

RNF-012: Recuperación ante Errores

Descripción: La aplicación debe manejar errores gracefully sin perder datos.

Criterios:

- Crash recovery: si la app cierra inesperadamente, al reabrirla recupera el estado
 - Rollback automático en caso de errores durante importación
 - Backup automático antes de operaciones destructivas (restaurar backup)
 - Log de errores para debugging (no visible al usuario final)
-

3.4 COMPATIBILIDAD

RNF-013: Versiones de Android

Descripción: La aplicación debe funcionar en versiones recientes de Android.

Especificación:

- **Versión mínima:** Android 8.0 (API 26) - Oreo
- **Versión objetivo:** Android 14 (API 34)
- **Cobertura estimada:** >85% de dispositivos activos

Justificación: Android 8.0+ permite usar características modernas manteniendo amplia compatibilidad.

RNF-014: Adaptabilidad de Pantalla

Descripción: La interfaz debe adaptarse a diferentes tamaños y orientaciones.

Criterios:

- Soporte para:
 - Teléfonos (4.5" - 7")
 - Tablets (7" - 13")
 - Orientaciones:
 - Portrait (preferida para uso en vivo)
 - Landscape (opcional para tablets)
 - Layout responsive usando ConstraintLayout
 - Elementos UI escalados apropiadamente según densidad de pantalla
-

RNF-015: Idioma

Descripción: La aplicación debe estar completamente en español.

Criterios:

- Toda la interfaz de usuario en español

- Mensajes de error en español
 - Documentación y ayuda en español
 - Formato de fechas según estándar argentino (DD/MM/YYYY)
 - Preparada para i18n (internacionalización) aunque solo se implemente español en v1
-

3.5 SEGURIDAD Y PRIVACIDAD

RNF-016: Privacidad de Datos

Descripción: Los datos del usuario deben permanecer privados y locales.

Criterios:

- Todos los datos almacenados localmente en el dispositivo
 - No se envían datos a servidores externos (excepto sincronización con BD central configurada por el usuario)
 - No se recolecta telemetría ni analytics
 - No se requiere creación de cuenta ni autenticación
 - Cumplimiento con principios de privacidad by design
-

RNF-017: Permisos Mínimos

Descripción: La aplicación debe solicitar solo permisos estrictamente necesarios.

Permisos requeridos:

- READ_EXTERNAL_STORAGE / READ_MEDIA_AUDIO - Para importar archivos
- WRITE_EXTERNAL_STORAGE - Para exportar backups (solo Android <10)
- WAKE_LOCK - Para mantener pantalla encendida en modo presentación
- INTERNET - Para sincronización y abrir enlaces (opcional)

Permisos NO requeridos:

- Ubicación
 - Cámara
 - Contactos
 - SMS/Llamadas
 - Micrófono (removido tras eliminar detección de acordes)
-

RNF-018: Validación de Datos

Descripción: Toda entrada de usuario y datos importados deben validarse.

Criterios:

- Validación de formato en campos de texto (longitud máxima, caracteres permitidos)
 - Sanitización de entrada para prevenir inyección SQL
 - Validación de archivos importados (formato, tamaño, estructura)
 - Manejo seguro de URLs externas
-

3.6 MANTENIBILIDAD

RNF-019: Arquitectura Limpia

Descripción: El código debe seguir principios de arquitectura limpia y mantenible.

Criterios:

- Arquitectura MVVM (Model-View-ViewModel)

- Separación clara de responsabilidades:
 - **UI Layer:** Compose/Views + ViewModels
 - **Domain Layer:** Use Cases / Interactors
 - **Data Layer:** Repository + Data Sources (Local DB)
 - Inyección de dependencias (Hilt o Koin)
 - Código testeable (unit tests para lógica de negocio)
-

RNF-020: Código Documentado

Descripción: El código debe estar adecuadamente documentado.

Criterios:

- KDoc para clases públicas y funciones complejas
 - Comentarios inline para lógica no obvia
 - README.md con instrucciones de setup
 - Documentación de arquitectura y decisiones técnicas
-

RNF-021: Control de Versiones

Descripción: El proyecto debe usar control de versiones efectivo.

Criterios:

- Git como sistema de control de versiones
 - Branching strategy definida (ej: GitFlow)
 - Commits descriptivos siguiendo convenciones
 - Tags para releases
-

3.7 ESCALABILIDAD

RNF-022: Crecimiento de Datos

Descripción: El sistema debe manejar crecimiento del repertorio sin degradación.

Criterios:

- Base de datos optimizada con índices apropiados
 - Queries eficientes (uso de índices en búsquedas)
 - Lazy loading de canciones en listas largas
 - Paginación si es necesario en listas de 500+ elementos
-

RNF-023: Actualizaciones de Esquema

Descripción: La base de datos debe soportar migraciones sin pérdida de datos.

Criterios:

- Sistema de migraciones de Room implementado
 - Versionado de esquema de BD
 - Migraciones testeadas
 - Rollback manual posible en caso de problemas
-

4. MODELO DE DATOS

4.1 Entidades Principales

Entidad: Cancion



kotlin

```
Cancion {
  id: Long (PK, AUTO_INCREMENT)
  titulo: String (NOT NULL, MAX 200)
  autor: String? (MAX 100)
  letra_con_acordes: String (NOT NULL)
  tonalidad_original: String (NOT NULL, MAX 10)
  tempo_bpm: Int? (RANGE 40-240)
  posicion_capo: Int (DEFAULT 0, RANGE 0-12)
  es_favorita: Boolean (DEFAULT false)
  contador_reproducciones: Int (DEFAULT 0)
  fecha_creacion: DateTime (NOT NULL)
  fecha_modificacion: DateTime (NOT NULL)
  notas: String? (MAX 500)
  enlaces_video: String? (JSON Array)
}
```

```
Índices:
- idx_titulo: titulo
- idx_favoritas: es_favorita
- idx_fecha_modificacion: fecha_modificacion
```

Entidad: Categoria



kotlin

```
Categoria {
  id: Long (PK, AUTO_INCREMENT)
  nombre: String (NOT NULL, UNIQUE, MAX 50)
  color: String? (Hex color, ej: "#FF5722")
  orden: Int (DEFAULT 0)
  es_predefinida: Boolean (DEFAULT false)
}
```

Índices:

- idx_nombre: nombre
- idx_orden: orden

Entidad: CancionCategoria (Tabla de relaciónMany-to-Many)



kotlin

```
CancionCategoria {
  id: Long (PK, AUTO_INCREMENT)
  cancion_id: Long (FK -> Cancion.id, ON DELETE CASCADE)
  categoria_id: Long (FK -> Categoria.id, ON DELETE CASCADE)
}
```

Índices:

- idx_cancion: cancion_id
- idx_categoria: categoria_id
- UNIQUE(cancion_id, categoria_id)

Entidad: Setlist



kotlin

```
Setlist {
  id: Long (PK, AUTO_INCREMENT)
  nombre: String (NOT NULL, MAX 100)
  fecha_evento: Date?
  notas: String? (MAX 300)
  fecha_creacion: DateTime (NOT NULL)
  fecha_modificacion: DateTime (NOT NULL)
}
```

- Índices:
- idx_nombre: nombre
 - idx_fecha_evento: fecha_evento

Entidad: SetlistCancion (Tabla de relación con configuración)



kotlin

```
SetlistCancion {
  id: Long (PK, AUTO_INCREMENT)
  setlist_id: Long (FK -> Setlist.id, ON DELETE CASCADE)
  cancion_id: Long (FK -> Cancion.id, ON DELETE CASCADE)
  orden: Int (NOT NULL)
  transposicion_semitonos: Int (DEFAULT 0, RANGE -11 to 11)
  capo_personalizado: Int? (RANGE 0-12)
}
```

- Índices:
- idx_setlist: setlist_id
 - idx_orden: (setlist_id, orden)
 - UNIQUE(setlist_id, cancion_id, orden)

Entidad: Configuracion (Key-Value Store)



kotlin

```
Configuracion {
  clave: String (PK, MAX 50)
  valor: String (NOT NULL)
  fecha_modificacion: DateTime (NOT NULL)
}
```

Configuraciones esperadas:

- "tema": "claro" | "nocturno"
- "tamanio_fuente": "pequeno" | "mediano" | "grande" | "muy_grande"
- "notacion_preferida": "americana" | "latina" | "ambas"
- "velocidad_scroll_default": "lento" | "medio" | "rapido"
- "factor_tempo": "1.0" (Float as String)
- "url_bd_central": "https://..."
- "sincronizar_auto": "true" | "false"
- "ultima_sincronizacion": "2025-10-14T10:30:00Z"

4.2 Relaciones



Cancion 1:N CancionCategoria N:1 Categoria
Cancion 1:N SetlistCancion N:1 Setlist

4.3 Categorías Predefinidas (Seed Data)

Al instalar la aplicación, se crean automáticamente estas categorías:



kotlin

```
val categoriasPredefinidas = listOf(  
    Categoria(nombre = "Entrada", color = "#4CAF50", orden = 1),  
    Categoria(nombre = "Meditación", color = "#2196F3", orden = 2),  
    Categoria(nombre = "Virgen María", color = "#E91E63", orden = 3),  
    Categoria(nombre = "Comunión", color = "#FFC107", orden = 4),  
    Categoria(nombre = "Ofertorio", color = "#FF9800", orden = 5),  
    Categoria(nombre = "Salida", color = "#9C27B0", orden = 6),  
    Categoria(nombre = "Adoración", color = "#FF5722", orden = 7),  
    Categoria(nombre = "Penitencial", color = "#795548", orden = 8),  
    Categoria(nombre = "Aleluya", color = "#FFEB3B", orden = 9)  
)
```

5. STACK TECNOLÓGICO RECOMENDADO

5.1 Core

- **Lenguaje:** Kotlin 1.9+
 - **IDE:** Android Studio Hedgehog (2023.1.1) o superior
 - **SDK Mínimo:** Android 8.0 (API 26)
 - **SDK Target:** Android 14 (API 34)
 - **Build System:** Gradle con Kotlin DSL
-

5.2 UI

- **Framework:** Jetpack Compose (preferido) o XML Views
 - **Material Design:** Material 3 (Material You)
 - **Navegación:** Navigation Component
-

5.3 Arquitectura

- **Patrón:** MVVM (Model-View-ViewModel)
 - **Inyección de Dependencias:** Hilt (recomendado) o Koin
 - **Async:** Kotlin Coroutines + Flow
-

5.4 Base de Datos

- **ORM:** Room Persistence Library
 - **Base de datos:** SQLite (incluido en Android)
-

5.5 Librerías Adicionales

- **Serialización JSON:** Kotlinx Serialization o Gson
- **Logging:** Timber
- **Testing:**
 - JUnit 4/5 para unit tests
 - Mockk para mocking
 - Espresso para UI tests

5.6 Sincronización (BD Central)

- **Networking:** Retrofit + OkHttp (para llamadas HTTP a BD central)
- **Formato de intercambio:** JSON
- **Opciones de BD Central:**
 - Firebase Realtime Database (simple, serverless)
 - REST API personalizada (PostgreSQL/MySQL en servidor propio)
 - Supabase (alternativa open-source a Firebase)

Nota: La arquitectura de la BD central debe definirse en conjunto con el equipo de backend.

6. CASOS DE USO PRIORITARIOS

6.1 Prioridad Crítica (MVP - Versión 1.0)

Estas funcionalidades son imprescindibles para el lanzamiento inicial:

1. **CRUD de Canciones**
 - Crear, editar, eliminar canciones
 - Insertar acordes sobre letra
 - Buscar canciones
 2. **Transposición y Capo**
 - Transponer acordes
 - Configurar capotraste
 3. **Modo Presentación Básico**
 - Visualización optimizada
 - Tamaño de fuente ajustable
 - Modo nocturno/claro
 - Mantener pantalla activa
 4. **Gestión de Setlists**
 - Crear setlists
 - Agregar/ordenar canciones
 - Reproducir setlist en modo presentación
 5. **Categorización**
 - Asignar categorías a canciones
 - Filtrar por categoría
 6. **Scroll Automático Inteligente**
 - Basado en tempo de canción
 - Controles play/pause
 7. **Almacenamiento Local**
 - Persistencia en SQLite
 - Funcionamiento offline
-

6.2 Prioridad Alta (Versión 1.1)

Funcionalidades importantes para completar la experiencia:

8. **Sincronización con BD Central**
 - Al abrir la app
 - Manual desde configuración
9. **Importar/Exportar**
 - Canciones individuales
 - Setlists completos
10. **Backup y Restauración**
 - Crear backup completo

- Restaurar desde backup

11. Favoritos y Búsqueda Avanzada

- Marcar favoritas
 - Búsqueda por texto en letra
-

6.3 Prioridad Media (Versión 1.2)

Mejoras y funcionalidades complementarias:

12. Estadísticas Básicas

- Contador de reproducciones
- Canciones más tocadas

13. Mejoras de UI/UX

- Animaciones pulidas
- Onboarding para nuevos usuarios

14. Notación Dual

- Cambio entre americana y latina
 - Mostrar ambas simultáneamente
-

6.4 Prioridad Baja (Versión 2.0+)

Funcionalidades adicionales para el futuro:

15. Importación desde Websites

- Web scraping de sitios de acordes
- Parser inteligente

16. Diagramas de Acordes

- Mostrar posiciones en diapasón
- Librería de acordes comunes

17. Compartir en Tiempo Real

- Sincronización live entre dispositivos
- Líder de alabanza controla todos los dispositivos

18. Metrónomo Integrado

- Para práctica individual
-

7. PLAN DE DESARROLLO SUGERIDO

Fase 1: Setup y Fundamentos (Sprint 1-2)

- Configuración del proyecto Android
- Setup de Room, Hilt, Navigation
- Modelo de datos implementado
- Seed data de categorías predefinidas
- UI básica con navegación

Entregable: App esqueleto navegable con BD funcional

Fase 2: CRUD y Gestión de Canciones (Sprint 3-4)

- Implementar RF-001 a RF-006
- Pantalla de lista de canciones
- Formulario crear/editar canción
- Vista de detalle de canción
- Lógica de transposición de acordes

- Lógica de capotraste

Entregable: Gestión completa de canciones funcional

Fase 3: Búsqueda y Organización (Sprint 5)

- Implementar RF-007 a RF-011
- Búsqueda en tiempo real
- Filtros por categoría
- Sistema de favoritos
- Listado alfabético con índice

Entregable: Organización y búsqueda completa

Fase 4: Setlists (Sprint 6-7)

- Implementar RF-012 a RF-017
- CRUD de setlists
- Drag & drop para reordenar
- Configuración personalizada por canción en setlist
- Navegación en setlist

Entregable: Sistema de setlists funcional

Fase 5: Modo Presentación (Sprint 8-9)

- Implementar RF-018 a RF-022
- UI de modo presentación
- Controles de visualización
- Scroll automático inteligente basado en tempo
- Mantener pantalla activa
- Temas claro/nocturno

Entregable: Modo presentación completo y pulido

Fase 6: Sincronización (Sprint 10-11)

- Implementar RF-023 a RF-024
- Arquitectura de BD central (coordinación con backend)
- Lógica de sincronización unidireccional
- Manejo de conflictos
- Indicadores de estado

Entregable: Sincronización con BD central funcional

Fase 7: Importar/Exportar y Backup (Sprint 12)

- Implementar RF-025 a RF-030
- Exportar/importar canciones
- Exportar/importar setlists
- Backup completo
- Restauración desde backup

Entregable: Sistema completo de portabilidad de datos

Fase 8: Configuración y Refinamiento (Sprint 13)

- Implementar RF-032 a RF-034
- Pantalla de configuración
- Preferencias de usuario
- Refinamiento de UI/UX
- Corrección de bugs

Entregable: MVP completo listo para testing beta

Fase 9: Testing y Optimización (Sprint 14-15)

- Testing exhaustivo
- Optimización de rendimiento
- Corrección de bugs críticos
- Documentación de usuario
- Preparación para release

Entregable: Versión 1.0 lista para producción

Fase 10 (Futuro): Funcionalidades Plus (Post-MVP)

- Implementar RF-031 (importar desde websites)
- Diagramas de acordes
- Estadísticas avanzadas
- Mejoras de UX basadas en feedback

Entregable: Versión 1.1+ con funcionalidades avanzadas

8. CRITERIOS DE ACEPTACIÓN GENERAL

8.1 Definición de "Hecho" (Definition of Done)

Una funcionalidad se considera completa cuando:

- ☒ Código implementado según requerimientos
 - ☒ Unit tests escritos y pasando (cobertura >70% para lógica de negocio)
 - ☒ UI tests para flujos críticos
 - ☒ Code review completado
 - ☒ Documentación técnica actualizada
 - ☒ Testing manual QA realizado
 - ☒ No hay bugs críticos ni bloqueantes
 - ☒ Cumple con estándares de código del proyecto
 - ☒ Merged a rama principal
-

8.2 Criterios de Calidad

Performance:

- Inicio de app < 3s
- Búsqueda < 500ms
- Navegación < 300ms
- Sin memory leaks
- Scroll a 60 FPS

Usabilidad:

- Interfaz intuitiva (usuarios prueban sin ayuda)
- Feedback visual para todas las acciones
- Manejo de errores claro

Confiabilidad:

- Tasa de crash < 1%
- Datos persisten correctamente
- Recuperación ante errores graceful

Compatibilidad:

- Funciona en Android 8.0+
- Adaptable a tablets y phones
- Portrait y landscape (donde aplique)

9. RIESGOS Y MITIGACIONES

Riesgo 1: Complejidad de Transposición de Acordes

Descripción: La lógica de transposición para acordes complejos puede ser propensa a errores.

Probabilidad: Media

Impacto: Alto

Mitigación:

- Implementar parser robusto de acordes
- Suite exhaustiva de unit tests con casos edge
- Validación con músicos reales

Riesgo 2: Sincronización con BD Central

Descripción: La sincronización puede ser compleja si la BD central no está bien definida.

Probabilidad: Media

Impacto: Medio

Mitigación:

- Definir contrato de API claro desde el inicio
- Implementar sincronización en fase tardía (MVP puede funcionar sin ella)
- Testing exhaustivo de escenarios de conflicto

Riesgo 3: Rendimiento con Repertorios Grandes

Descripción: Con 500+ canciones, las búsquedas y listados pueden ser lentos.

Probabilidad: Baja

Impacto: Medio

Mitigación:

- Optimizar queries con índices apropiados

- Implementar paginación o lazy loading
 - Testing de performance desde etapas tempranas
-

Riesgo 4: Scroll Automático no Sincronizado con Tempo

Descripción: El cálculo de velocidad de scroll basado en tempo puede no ser preciso.
Probabilidad: Media
Impacto: Medio
Mitigación:

- Permitir ajuste manual de velocidad sobre el cálculo automático
 - Iteración con usuarios reales para calibrar algoritmo
 - Opción de scroll manual tradicional siempre disponible
-

Riesgo 5: Importación desde Websites (Función Plus)

Descripción: Los sitios web pueden cambiar su estructura HTML, rompiendo el scraper.
Probabilidad: Alta
Impacto: Bajo (es función Plus)
Mitigación:

- Implementar adaptadores por sitio
 - Manejo robusto de errores
 - Documentar que es funcionalidad "best effort"
 - No es crítica para MVP
-

10. MÉTRICAS DE ÉXITO

10.1 Métricas Técnicas

- **Tasa de crashes:** < 1% de sesiones
 - **Tiempo de inicio:** < 3 segundos (P95)
 - **Tiempo de respuesta de búsqueda:** < 500ms (P95)
 - **Cobertura de tests:** > 70% para lógica de negocio
 - **Deuda técnica:** Manejable (sin code smells críticos)
-

10.2 Métricas de Producto (Post-lanzamiento)

- **Adopción:** X% del grupo musical usando la app regularmente
 - **Retención:** >80% de usuarios activos tras primer mes
 - **Repertorio:** Promedio de X canciones por usuario
 - **Setlists:** Promedio de X setlists creados por usuario
 - **Satisfacción:** Puntuación >4/5 en encuesta de usuarios
-

10.3 Indicadores de Calidad

- **Feedback de usuarios:** Positivo en aspectos de usabilidad
 - **Bugs reportados:** < 5 bugs críticos post-lanzamiento
 - **Tiempo de resolución:** Bugs críticos resueltos en < 48h
-

11. GLOSARIO

Acorde: Combinación de notas musicales tocadas simultáneamente. En esta app, se refiere a la notación escrita (ej: C, Am7, F#m).

BPM (Beats Per Minute): Tempo musical, número de pulsos por minuto.

Capo/Capotraste: Dispositivo que se coloca en el mástil de la guitarra para cambiar la afinación sin alterar las posiciones de los acordes.

Notación Americana: Sistema de notación de acordes usando letras: C, D, E, F, G, A, B.

Notación Latina: Sistema de notación de acordes usando sílabas: Do, Re, Mi, Fa, Sol, La, Si.

Setlist: Lista de canciones ordenadas para un evento específico (misa, bautismo, etc.).

Transposición: Cambio de tonalidad de una canción, moviendo todos los acordes el mismo número de semitonos.

Semitono: Distancia más pequeña entre dos notas en la música occidental (ej: de C a C#).

Tonalidad: Nota fundamental sobre la que se construye una canción (ej: "Canción en Sol mayor").

Scroll Automático: Desplazamiento automático de la pantalla durante la reproducción de una canción.

Modo Presentación: Vista optimizada para visualizar canciones durante la ejecución, con controles mínimos y texto grande.

BD Central: Base de datos centralizada desde donde se sincronizan las canciones al repertorio local de cada usuario.

12. APÉNDICES

Apéndice A: Formato de Archivo de Exportación/Importación

Formato de Canción Individual (JSON)



json

```
{
  "version": "1.0",
  "tipo": "cancion",
  "fecha_exportacion": "2025-10-14T15:30:00Z",
  "cancion": {
    "titulo": "Ejemplo de Canción",
    "autor": "Autor Ejemplo",
    "letra_con_acordes": "  C      Am\nEjemplo de letra...",
    "tonalidad_original": "C",
    "tempo_bpm": 120,
    "posicion_capo": 0,
    "notas": "Canta Juan en el estribillo",
    "enlaces_video": [
      "https://youtube.com/watch?v=ejemplo"
    ],
    "categorias": ["Entrada", "Adoración"]
  }
}
```

Formato de Setlist Completo (JSON)



json

```
{
  "version": "1.0",
  "tipo": "setlist",
  "fecha_exportacion": "2025-10-14T15:30:00Z",
  "setlist": {
    "nombre": "Misa Domingo 20/10/2025",
    "fecha_evento": "2025-10-20",
    "notas": "Misa de 10:00 AM",
    "canciones": [
      {
        "orden": 1,
        "transposicion_semitonos": 0,
        "capo_personalizado": 2,
        "cancion": {
          "titulo": "Canción de Entrada",
          "autor": "Autor Ejemplo",
          "letra_con_acordes": "...",
          "tonalidad_original": "G",
          "tempo_bpm": 90,
          "posicion_capo": 0,
          "notas": "",
          "enlaces_video": [],
          "categorias": ["Entrada"]
        }
      },
      {
        "orden": 2,
        "transposicion_semitonos": -2,
        "capo_personalizado": null,
        "cancion": {
          "titulo": "Canción de Meditación",
          "autor": "Otro Autor",
          "letra_con_acordes": "...",
          "tonalidad_original": "C",
          "tempo_bpm": 72,
          "posicion_capo": 0,
          "notas": "",
          "enlaces_video": [],
          "categorias": ["Meditación"]
        }
      }
    ]
  }
}
```



```
}  
}
```

Formato de Backup Completo (ZIP con JSON)

Estructura del archivo ZIP:



```
CancioneroBackup_20251014_153000.zip  
├── metadata.json  
├── canciones.json  
├── categorias.json  
├── setlists.json  
└── configuracion.json
```

metadata.json:



json

```
{  
  "version_backup": "1.0",  
  "fecha_creacion": "2025-10-14T15:30:00Z",  
  "version_app": "1.0.0",  
  "total_canciones": 150,  
  "total_categorias": 12,  
  "total_setlists": 8  
}
```

canciones.json: Array de todas las canciones con el formato individual mostrado arriba.

categorias.json:



json

```
[
  {
    "nombre": "Entrada",
    "color": "#4CAF50",
    "orden": 1,
    "es_predefinida": true
  },
  {
    "nombre": "Mi Categoría",
    "color": "#9C27B0",
    "orden": 10,
    "es_predefinida": false
  }
]
```

setlists.json: Array de todos los setlists con el formato completo mostrado arriba.

configuracion.json:



json

```
{
  "tema": "nocturno",
  "tamano_fuente": "grande",
  "notacion_preferida": "americana",
  "velocidad_scroll_default": "medio",
  "factor_tempo": "1.0",
  "url_bd_central": "https://ejemplo.com/api",
  "sincronizar_auto": "true"
}
```

Apéndice B: Algoritmo de Transposición de Acordes

Círculo de Quintas y Transposición

Notación Americana:



Orden cromático: C, C#/Db, D, D#/Eb, E, F, F#/Gb, G, G#/Ab, A, A#/Bb, B

Índices: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11

Notación Latina:



Orden cromático: Do, Do#/Reb, Re, Re#/Mib, Mi, Fa, Fa#/Solb, Sol, Sol#/Lab, La, La#/Sib, Si
Índices: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11

Lógica de Transposición



kotlin

```

fun transponerAcorde(acorde: String, semitonos: Int): String {
    // 1. Parsear el acorde para extraer:
    //   - Nota base (C, D, E, etc.)
    //   - Modificador (# o b)
    //   - Tipo (m, 7, maj7, sus4, etc.)

    val regex = """"^([A-G]|Do|Re|Mi|Fa|Sol|La|Si)(#|b)?(.*?)$""".toRegex()
    val match = regex.find(acorde) ?: return acorde

    val (notaBase, modificador, extension) = match.destructured

    // 2. Convertir nota base a índice (0-11)
    var indice = notaAIndice(notaBase, modificador)

    // 3. Aplicar transposición (módulo 12 para mantener en rango)
    indice = (indice + semitonos + 12) % 12

    // 4. Convertir índice de vuelta a nota
    val nuevaNota = indiceANota(indice, preferenciaNotacion())

    // 5. Reconstruir el acorde
    return nuevaNota + extension
}

```

```

fun notaAIndice(nota: String, modificador: String): Int {
    val baseAmericana = mapOf(
        "C" to 0, "D" to 2, "E" to 4, "F" to 5,
        "G" to 7, "A" to 9, "B" to 11
    )
    val baseLatina = mapOf(
        "Do" to 0, "Re" to 2, "Mi" to 4, "Fa" to 5,
        "Sol" to 7, "La" to 9, "Si" to 11
    )

    var indice = baseAmericana[nota] ?: baseLatina[nota] ?: 0

    when (modificador) {
        "#" -> indice += 1
        "b" -> indice -= 1
    }

    return (indice + 12) % 12
}

```

```

fun indiceANota(indice: Int, notacion: TipoNotacion): String {
    val notasAmericanas = arrayOf(
        "C", "C#", "D", "D#", "E", "F",
        "F#", "G", "G#", "A", "A#", "B"
    )
    val notasLatinas = arrayOf(
        "Do", "Do#", "Re", "Re#", "Mi", "Fa",
        "Fa#", "Sol", "Sol#", "La", "La#", "Si"
    )

    return when (notacion) {
        TipoNotacion.AMERICANA -> notasAmericanas[indice]
        TipoNotacion.LATINA -> notasLatinas[indice]
    }
}

```

Ejemplos de Transposición

Transponer +2 semitonos:

- C → D
- Am → Bm
- F#m7 → G#m7
- Dsus4 → Esus4
- Bb → C

Transponer -3 semitonos:

- G → E
- Em7 → C#m7
- A → F#
- Cmaj7 → Amaj7

Apéndice C: Algoritmo de Scroll Automático Inteligente

Cálculo de Velocidad basado en Tempo



kotlin

```

data class ConfiguracionScroll(
    val tempoBPM: Int?,           // Tempo de la canción
    val longitudTexto: Int,       // Líneas totales de la canción
    val alturaPantalla: Int,      // Alto visible en pixels
    val factorAjuste: Float = 1.0f // Factor de ajuste del usuario (0.5x - 2.0x)
)

fun calcularVelocidadScroll(config: ConfiguracionScroll): Float {
    val tempoBPM = config.tempoBPM ?: return velocidadPorDefecto()

    // Duración estimada de la canción en segundos
    // Suposición: ~4 compases por línea, tempo en negras (4/4)
    val compasesPorLinea = 2.0
    val negrasCompas = 4.0
    val duracionEstimada = (config.longitudTexto * compasesPorLinea * negrasCompas) /
        (tempoBPM / 60.0)

    // Velocidad en pixels por segundo para completar el scroll
    val velocidadBase = (config.longitudTexto * alturaLineaPromedio()) / duracionEstimada

    // Aplicar factor de ajuste del usuario
    return velocidadBase * config.factorAjuste
}

fun velocidadPorDefecto(): Float {
    // Velocidad media configurable (ej: 30 pixels/segundo)
    return configuracion.velocidadScrollDefault.toFloat()
}

fun alturaLineaPromedio(): Float {
    // Altura promedio de una línea según tamaño de fuente actual
    return when (configuracion.tamanoFuente) {
        "pequeno" -> 40f
        "mediano" -> 50f
        "grande" -> 60f
        "muy_grande" -> 70f
        else -> 50f
    }
}

```

Implementación del Scroll



kotlin

```

class ScrollAutoController(
    private val scrollView: ScrollView,
    private val config: ConfiguracionScroll
) {
    private var isScrolling = false
    private var scrollJob: Job? = null

    fun iniciar() {
        if (isScrolling) return

        isScrolling = true
        val velocidad = calcularVelocidadScroll(config)

        scrollJob = CoroutineScope(Dispatchers.Main).launch {
            val incrementoPorFrame = velocidad / 60f // 60 FPS

            while (isScrolling && !scrollView.isAtBottom()) {
                scrollView.scrollBy(0, incrementoPorFrame.toInt())
                delay(16) // ~60 FPS
            }

            // Al llegar al final, detener automáticamente
            if (scrollView.isAtBottom()) {
                detener()
            }
        }
    }

    fun pausar() {
        isScrolling = false
        scrollJob?.cancel()
    }

    fun detener() {
        pausar()
        scrollView.scrollTo(0, 0) // Volver al inicio
    }

    fun ajustarVelocidad(nuevoFactor: Float) {
        config.factorAjuste = nuevoFactor
        if (isScrolling) {
            pausar()
            iniciar() // Reiniciar con nueva velocidad
        }
    }
}

```



```
}  
}
```

Apéndice D: Estructura de la BD Central (Sugerencia)

Opción 1: Firebase Realtime Database

Estructura JSON sugerida:



json

```
{
  "canciones": {
    "cancion_id_1": {
      "titulo": "Ejemplo",
      "autor": "Autor",
      "letra_con_acordes": "...",
      "tonalidad_original": "C",
      "tempo_bpm": 120,
      "posicion_capo": 0,
      "notas": "",
      "enlaces_video": ["url1", "url2"],
      "categorias": ["Entrada", "Adoración"],
      "fecha_modificacion": "2025-10-14T15:30:00Z"
    },
    "cancion_id_2": { ... }
  },
  "categorias": {
    "categoria_id_1": {
      "nombre": "Entrada",
      "color": "#4CAF50",
      "orden": 1
    }
  },
  "setlists": {
    "setlist_id_1": {
      "nombre": "Misa Domingo",
      "fecha_evento": "2025-10-20",
      "notas": "",
      "canciones": [
        {
          "cancion_id": "cancion_id_1",
          "orden": 1,
          "transposicion": 0,
          "capo": 2
        }
      ],
      "fecha_modificacion": "2025-10-14T15:30:00Z"
    }
  },
  "metadata": {
    "ultima_actualizacion": "2025-10-14T15:30:00Z",
    "version_esquema": "1.0"
  }
}
```

```
}  
}
```

Reglas de seguridad Firebase:



json

```
{  
  "rules": {  
    "canciones": {  
      ".read": true,  
      ".write": "auth != null"  
    },  
    "categorias": {  
      ".read": true,  
      ".write": "auth != null"  
    },  
    "setlists": {  
      ".read": true,  
      ".write": "auth != null"  
    }  
  }  
}
```

Opción 2: REST API Personalizada

Endpoints sugeridos:



GET /api/v1/sync

Response:

```
{  
  "ultima_actualizacion": "2025-10-14T15:30:00Z",  
  "canciones": [...],  
  "categorias": [...],  
  "setlists": [...]  
}
```

GET /api/v1/sync?desde=2025-10-01T00:00:00Z

Response: Solo elementos modificados después de la fecha especificada

GET /api/v1/canciones

GET /api/v1/canciones/{id}

GET /api/v1/categorias

GET /api/v1/setlists

Autenticación: API Key simple o JWT si se requiere autenticación de usuarios.

Apéndice E: Wireframes Conceptuales (Descripción Textual)

Pantalla 1: Lista Principal de Canciones



+-----+	
[☰] Canciones	[🔍] [+]
+-----+	
Buscar...	
+-----+	
A	
• Alabado Sea el Señor	★
Entrada, Adoración	
• Aleluya de Haendel	
Aleluya	
+-----+	
B	
• Bendito Seas Señor	
Comunión	
+-----+	
[Índice A-Z lateral]	
+-----+	
[🎵] Canciones	[📋] Setlists
[📁] Categorías	[⚙️] Config
+-----+	

Pantalla 2: Vista de Detalle de Canción



+-----+

| [←] Alabado Sea el Señor [:] |

+-----+

| Autor: Juan Pérez |

| [Entrada] [Adoración] |

+-----+

| Tono: G Capo: 2 BPM: 90 |

| [-1] [Tono] [+1] [🔺 Capo] |

+-----+

| G Em |

| Alabado sea el Señor |

| C D |

| Por su amor y compasión |

| |

| [resto de la letra...] |

+-----+

| Notas: Canta María el estribillo |

| 📺 YouTube: [Ver video] |

+-----+

| [📝 Editar] [▶ Presentación] |

+-----+

Pantalla 3: Modo Presentación



+-----+
 | [X] |
 | |
 | Alabado Sea el Señor |
 | Tono: G (+2) | Capo: 2 |
 | |
 | G Em |
 | Alabado sea el Señor |
 | C D |
 | Por su amor y compasión |
 | |
 | [resto de letra grande] |
 | |
 | |
 | [◀] 3/8 [▶] [▶] [abc] [🧐] [±] |
 +-----+

Controles:

- [◀][▶]: Anterior/Siguiente en setlist
- [▶]: Play/Pause scroll automático
- [abc]: Ajustar tamaño de fuente
- [🌞]: Cambiar tema (claro/nocturno)
-

Pantalla 4: Lista de Setlists



+-----+		
	[☰] Setlists	[+]
+-----+		
	📄 Misa Domingo 20/10/2025	
	8 canciones 20/10/2025	
	[▶️ Reproducir]	
+-----+		
	📄 Bautismo Juan	
	5 canciones 15/11/2025	
	[▶️ Reproducir]	
+-----+		
	📄 Misa de Navidad	
	12 canciones 25/12/2025	
	[▶️ Reproducir]	
+-----+		
	[🎵 Canciones] [📄 Setlists]	
	[📁 Categorías] [⚙️ Config]	
+-----+		

Pantalla 5: Detalle de Setlist



+-----+		
	[←] Misa Domingo 20/10	[:]
+-----+		
	Fecha: 20/10/2025	
	Notas: Misa de 10:00 AM	
+-----+		
	1. [☰] Alabado Sea el Señor	
	G (+2) Capo: 2	
	2. [☰] Canto de Meditación	
	C (0) Sin capo	
	3. [☰] Ave María	
	D (-1) Capo: 3	
	[+ Agregar canción]	
+-----+		
	[✏️ Editar] [▶️ Reproducir]	
+-----+		

Pantalla 6: Configuración



+-----+

| [←] Configuración |

+-----+

| VISUALIZACIÓN |

| Tema: ☒ Nocturno ☐ Claro |

| Tamaño fuente: [-] Mediano [+] |

| Notación: ☒ Americana ☐ Latina |

| ☐ Ambas |

+-----+

| SCROLL AUTOMÁTICO |

| Velocidad default: Medio |

| Factor tempo: 1.0x [slider] |

+-----+

| SINCRONIZACIÓN |

| URL BD Central: [.....] |

| ☒ Sincronizar al abrir |

| Última sync: 14/10 15:30 |

| [ Sincronizar ahora] |

+-----+

| DATOS |

| [ Crear Backup] |

| [ Restaurar Backup] |

| [ Importar Canción] |

+-----+

| Versión: 1.0.0 |

+-----+

Apéndice F: Checklist de Testing Pre-Release

Testing Funcional

Canciones:

- ☐ Crear canción con todos los campos
- ☐ Editar canción existente
- ☐ Eliminar canción (con confirmación)
- ☐ Insertar acordes sobre letra
- ☐ Transponer acordes en ambas direcciones
- ☐ Configurar capo (0-12)
- ☐ Transponer + capo juntos
- ☐ Guardar enlaces de video

- ☐ Validación de campos obligatorios

Búsqueda y Filtros:

- ☐ Búsqueda en tiempo real funciona
- ☐ Búsqueda en título
- ☐ Búsqueda en autor
- ☐ Búsqueda en letra
- ☐ Búsqueda en notas
- ☐ Filtro por categoría (una)
- ☐ Filtro por categorías múltiples
- ☐ Filtro + búsqueda combinados
- ☐ Limpiar filtros
- ☐ Favoritos

Setlists:

- ☐ Crear setlist
- ☐ Agregar canciones al setlist
- ☐ Reordenar canciones (drag & drop)
- ☐ Editar setlist
- ☐ Eliminar canción de setlist
- ☐ Eliminar setlist completo
- ☐ Transposición personalizada en setlist
- ☐ Capo personalizado en setlist
- ☐ Reproducir setlist completo
- ☐ Navegación anterior/siguiente en setlist

Modo Presentación:

- ☐ Entrar en modo presentación
- ☐ Salir de modo presentación
- ☐ Cambiar tamaño de fuente en vivo
- ☐ Cambiar tema claro/nocturno
- ☐ Transponer en modo presentación
- ☐ Scroll automático play/pause
- ☐ Scroll automático con tempo
- ☐ Scroll automático sin tempo
- ☐ Ajustar velocidad de scroll
- ☐ Pantalla se mantiene encendida
- ☐ Navegación en setlist funciona

Sincronización:

- ☐ Sincronización al abrir app (con conexión)
- ☐ Comportamiento sin conexión
- ☐ Sincronización manual
- ☐ Indicador de estado de sync
- ☐ Manejo de errores de conexión

Importar/Exportar:

- ☐ Exportar canción individual
- ☐ Importar canción (nueva)
- ☐ Importar canción (duplicada - sobrescribir)
- ☐ Importar canción (duplicada - nueva)

- ☐ Exportar setlist completo
- ☐ Importar setlist completo
- ☐ Crear backup completo
- ☐ Restaurar desde backup
- ☐ Validación de formatos incorrectos

Configuración:

- ☐ Cambiar tema (persiste)
 - ☐ Cambiar tamaño fuente (persiste)
 - ☐ Cambiar notación (persiste)
 - ☐ Configurar velocidad scroll (persiste)
 - ☐ Configurar URL BD central
 - ☐ Toggle sincronización automática
-

Testing No Funcional

Rendimiento:

- ☐ Inicio de app < 3s
- ☐ Búsqueda < 500ms con 500 canciones
- ☐ Navegación entre vistas fluida
- ☐ Scroll automático a 60 FPS
- ☐ No memory leaks

Usabilidad:

- ☐ Usuario nuevo completa tareas sin ayuda
- ☐ Botones suficientemente grandes (48dp+)
- ☐ Feedback visual para todas las acciones
- ☐ Mensajes de error claros

Compatibilidad:

- ☐ Funciona en Android 8.0
- ☐ Funciona en Android 14
- ☐ Funciona en teléfonos (5")
- ☐ Funciona en tablets (10")
- ☐ Portrait y landscape (donde aplique)

Confiabilidad:

- ☐ Datos persisten tras cierre
- ☐ Datos persisten tras reinicio
- ☐ Recuperación ante crash
- ☐ Sin corrupción de BD en escenarios adversos

Offline:

- ☐ 100% funcional sin internet (excepto sync)
 - ☐ No errores de conectividad molestos
-

Testing de Regresión

- ☐ Todas las funcionalidades previas siguen funcionando
- ☐ No hay nuevos bugs introducidos
- ☐ Performance no ha degradado

13. CONTACTO Y APROBACIONES

Documento preparado por: [Equipo de Análisis]
Fecha: 14 de Octubre de 2025
Versión: 1.0

Aprobaciones requeridas:

Rol	Nombre	Firma	Fecha
Product Owner			
Tech Lead			
QA Lead			

Historial de cambios:

Versión	Fecha	Autor	Cambios
1.0	14/10/2025	Equipo Análisis	Versión inicial aprobada

14. REFERENCIAS

- [Material Design 3 Guidelines](#)
- [Android Developers - Room Persistence Library](#)
- [Jetpack Compose Documentation](#)
- [Kotlin Coroutines Guide](#)
- [WCAG Accessibility Guidelines](#)

FIN DEL DOCUMENTO DE REQUERIMIENTOS

Este documento debe ser revisado y actualizado conforme el proyecto evolucione. Cualquier cambio significativo en los requerimientos debe ser documentado y aprobado por los stakeholders correspondientes.